

Объектно-ориентированное программирование

Никита Остроухов

Кафедра информатики МФТИ

Архитектура графических приложений

2D игра

Идеальный газ

Hacker Typer

2D игра

Игрок и враги

Отрисовка спрайтов

Перемещение

Соударение

Правила игры

Взаимодействие

Идеальный газ

Hacker Typer

2D игра

Игрок и враги

Отрисовка спрайтов

Перемещение

Соударение

Правила игры

Взаимодействие

Идеальный газ

Частицы

Отрисовка шаров

Перемещение

Соударения

Гравитация

Hacker Typer

2D игра

Игрок и враги

Отрисовка спрайтов

Перемещение

Соударение

Правила игры

Взаимодействие

Идеальный газ

Частицы

Отрисовка шаров

Перемещение

Соударения

Гравитация

Hacker Typer

Текст

Отрисовка текста

Правила

Взаимодействие

Архитектура приложения

Игрок и враги

Частицы

Текст

Отрисовка спрайтов

Отрисовка шаров

Отрисовка текста

Перемещение

Соударение

Гравитация

Правила игра

Правила взаимодействия

Взаимодействие

Архитектура приложения

Игрок и враги
Частицы
Текст



Хранение данных

Отрисовка спрайтов
Отрисовка шаров
Отрисовка текста

Перемещение
Соударение
Гравитация

Правила игра
Правила взаимодействия

Взаимодействие

Архитектура приложения

Игрок и враги
Частицы
Текст



Хранение данных

Отрисовка спрайтов
Отрисовка шаров
Отрисовка текста



Графика приложения

Перемещение
Соударение
Гравитация

Правила игра
Правила взаимодействия

Взаимодействие

Архитектура приложения

Игрок и враги
Частицы
Текст



Хранение данных

Отрисовка спрайтов
Отрисовка шаров
Отрисовка текста



Графика приложения

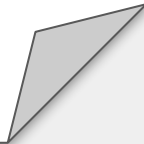
Перемещение
Соударение
Гравитация



Физика приложения

Правила игра
Правила взаимодействия

Взаимодействие



Архитектура приложения

Игрок и враги
Частицы
Текст



Хранение данных

Отрисовка спрайтов
Отрисовка шаров
Отрисовка текста



Графика приложения

Перемещение
Соударение
Гравитация



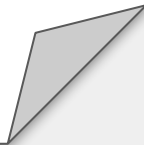
Физика приложения

Правила игра
Правила взаимодействия



Логика приложения

Взаимодействие



Архитектура приложения

Игрок и враги
Частицы
Текст



Хранение данных

Отрисовка спрайтов
Отрисовка шаров
Отрисовка текста



Графика приложения

Перемещение
Соударение
Гравитация



Физика приложения

Правила игра
Правила взаимодействия



Логика приложения

Взаимодействие



Интерфейс приложения

Архитектура приложения

Интерфейс взаимодействия

Хранение данных

**Менеджер
графики**

**Менеджер
физики**

**Менеджер
логики**

Архитектура приложения

Пользователь



Интерфейс взаимодействия

Хранение данных

Менеджер
графики

Менеджер
физики

Менеджер
логики

Архитектура приложения

Пользователь



Интерфейс взаимодействия



Хранение данных

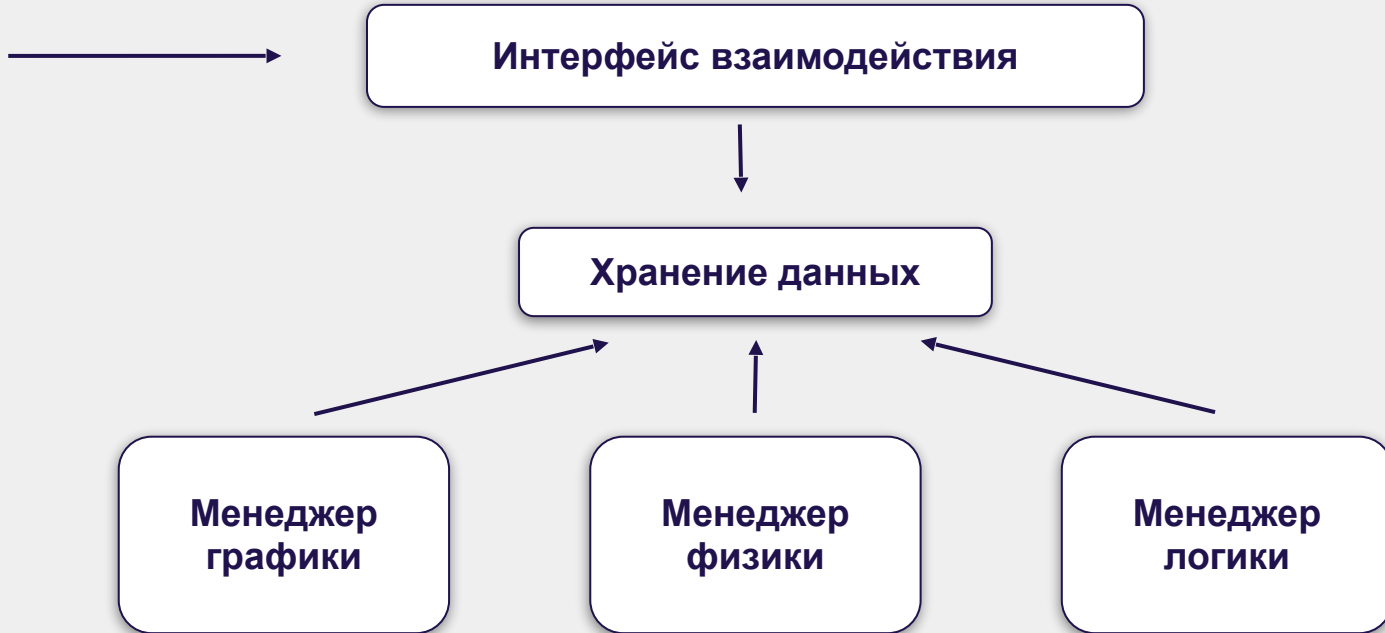
Менеджер
графики

Менеджер
физики

Менеджер
логики

Архитектура приложения

Пользователь



Данные как ядро приложения

Важно как хранить данные

Способы хранения

Хранение множества объектов:

Способы хранения

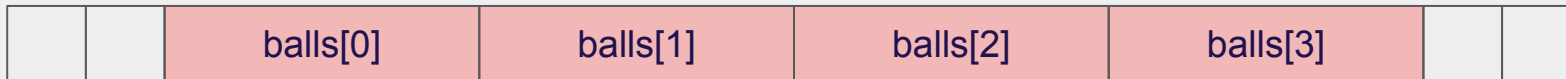
Хранение множества объектов:

- **Массив**

Оперативная память

		balls[0]	balls[1]	balls[2]	balls[3]		
--	--	----------	----------	----------	----------	--	--

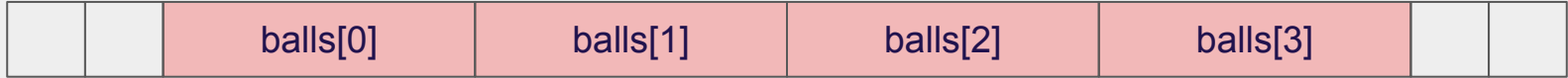
Оперативная память



Плюсы:

- **Итерация** по массиву эффективна за счет **кэша** процессора
- Доступ к любому элементу за **константное** время

Оперативная память



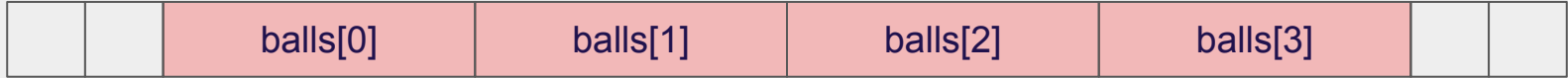
Плюсы:

- **Итерация** по массиву эффективна за счет **кэша** процессора
- Доступ к любому элементу за **константное** время

Минусы:

- Требуется выделение **подряд лежащей** памяти
- **Реаллокация** массива при его увеличении/уменьшении

Оперативная память



Плюсы:

- **Итерация** по массиву эффективна за счет **кэша** процессора
- Доступ к любому элементу за **константное** время

Минусы:

- Требуется выделение **подряд лежащей** памяти
- **Реаллокация** массива при его увеличении/уменьшении

Способы хранения

Хранение множества объектов:

- Массив
- ...

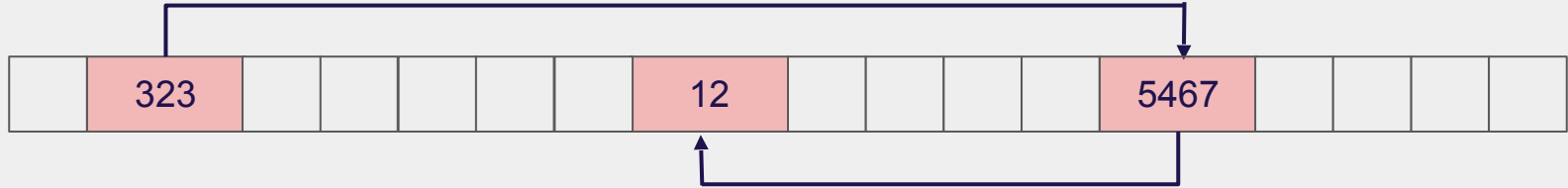
Список

Оперативная память

	323						12					5467				
--	-----	--	--	--	--	--	----	--	--	--	--	------	--	--	--	--

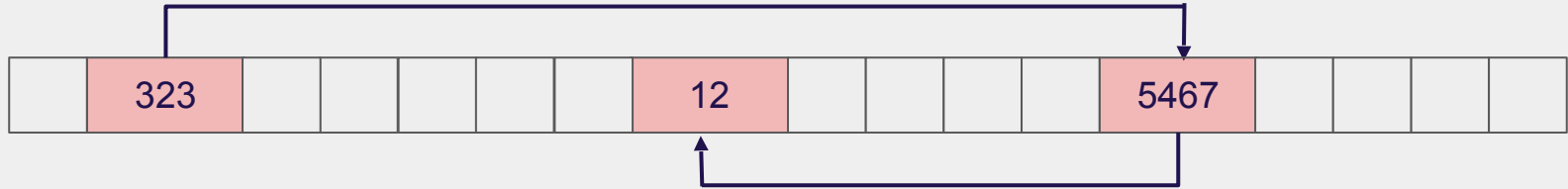
Список: 323 546 12

Оперативная память

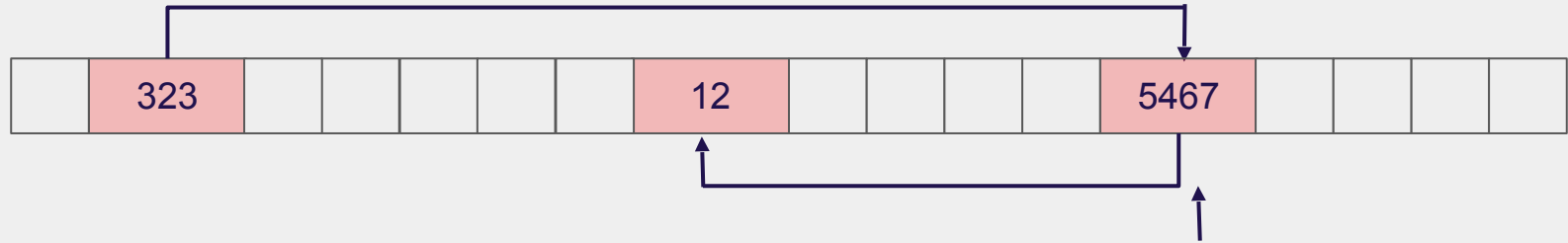


Список: 323 546 12

Оперативная память



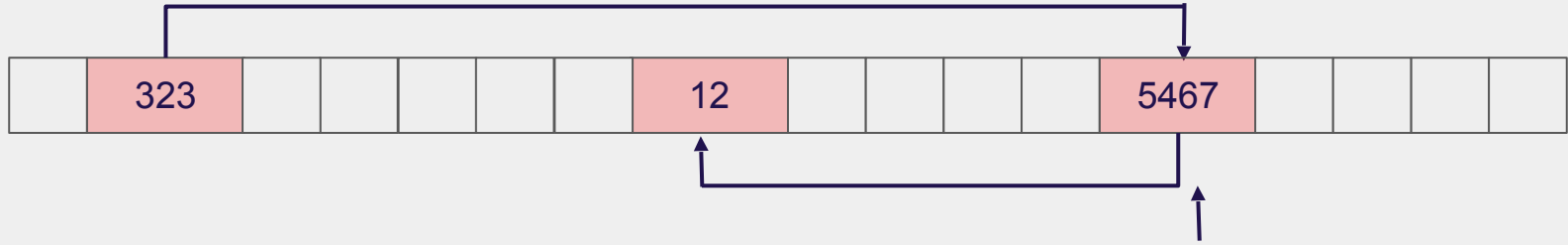
Оперативная память



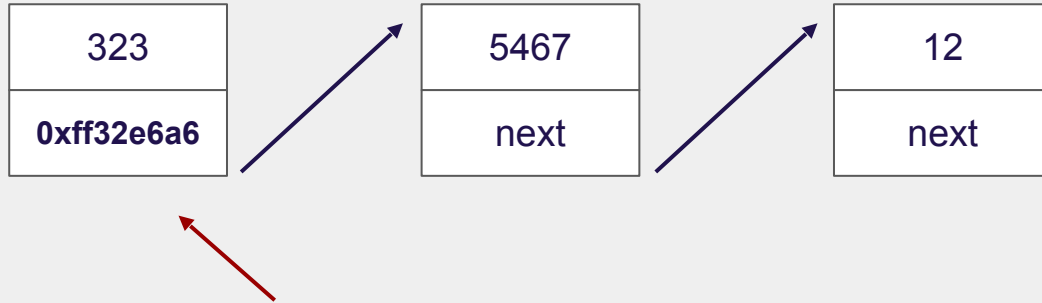
0xff32e6a6



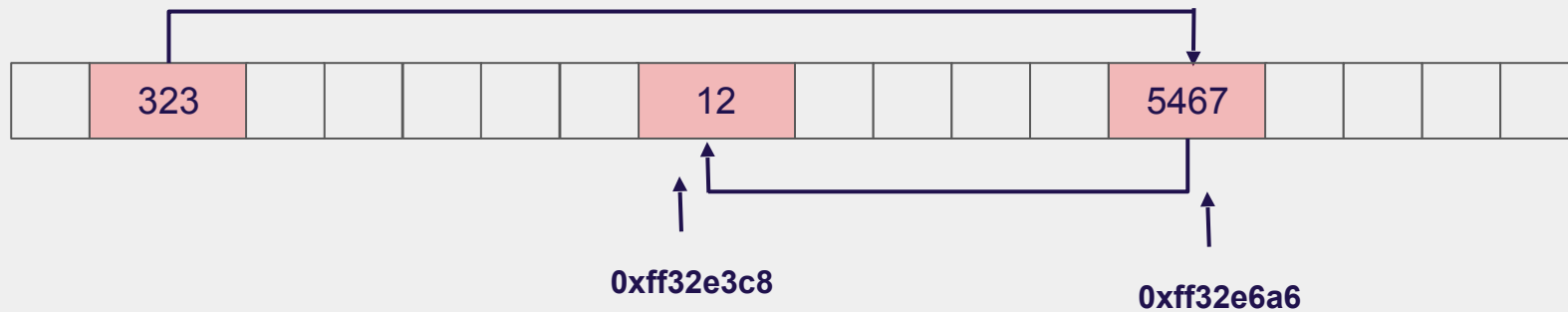
Оперативная память



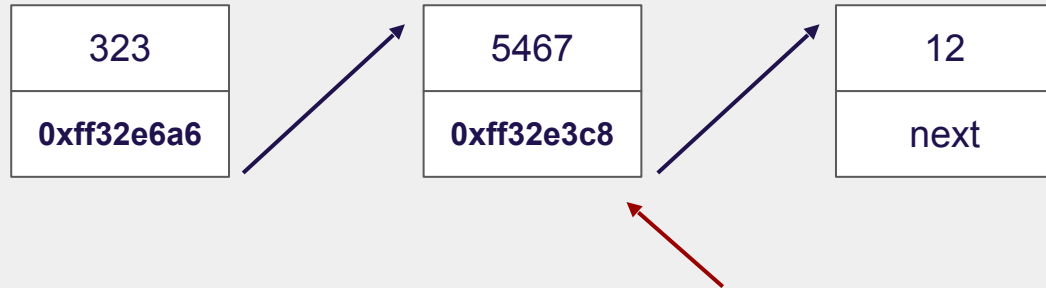
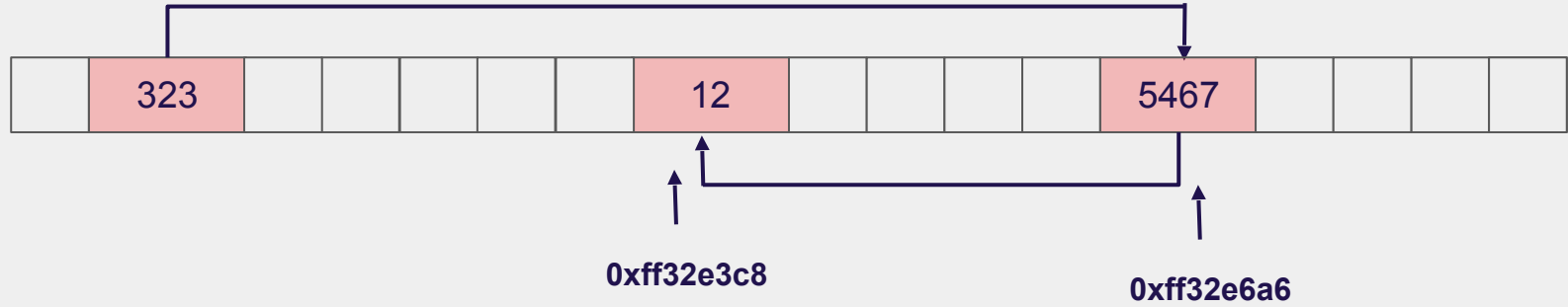
0xff32e6a6



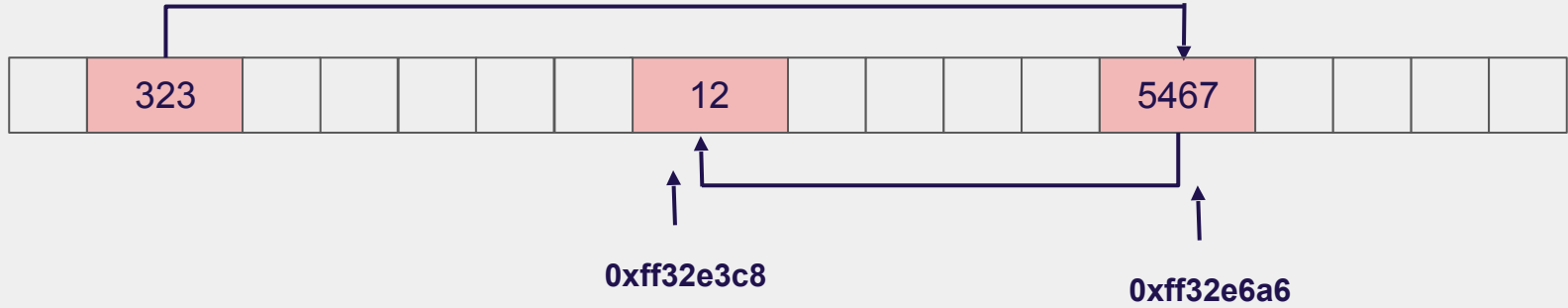
Оперативная память



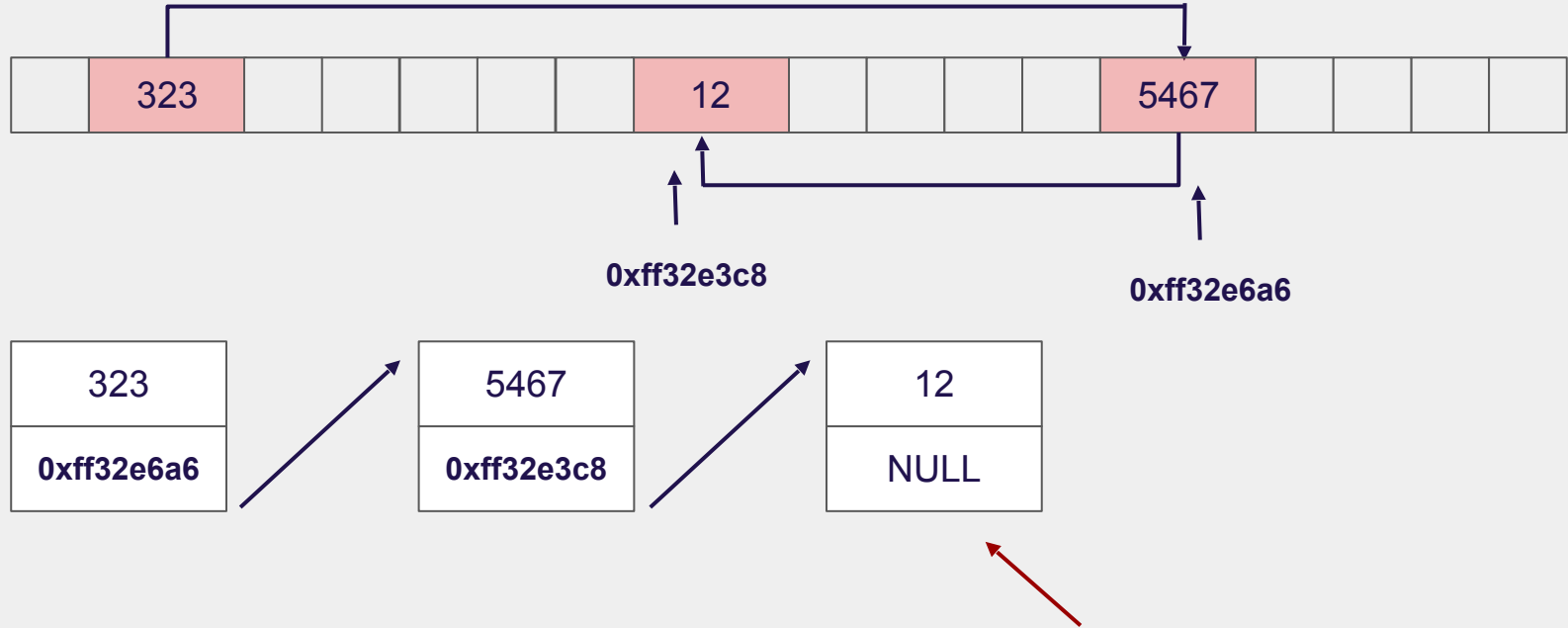
Оперативная память



Оперативная память

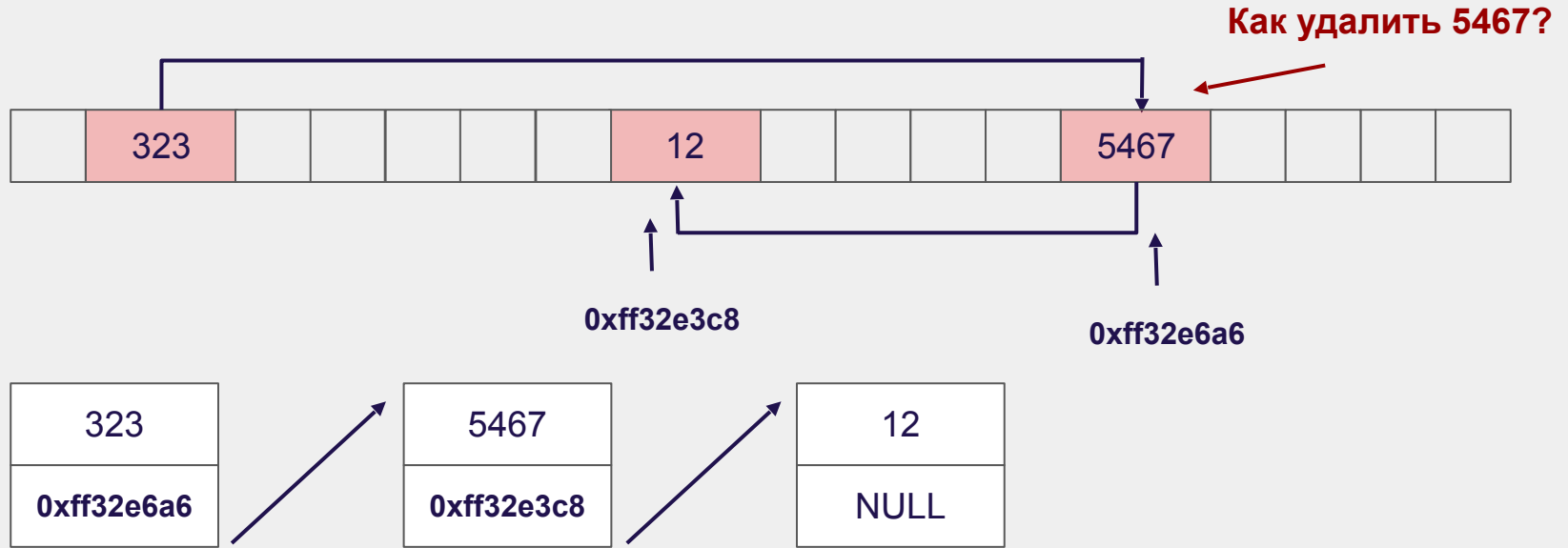


Оперативная память

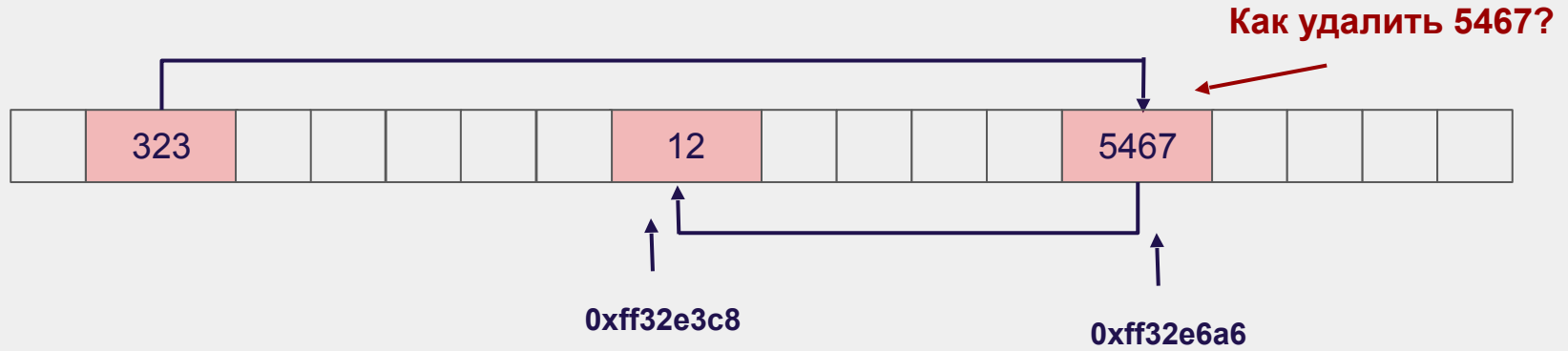


Удаление элемента

Оперативная память

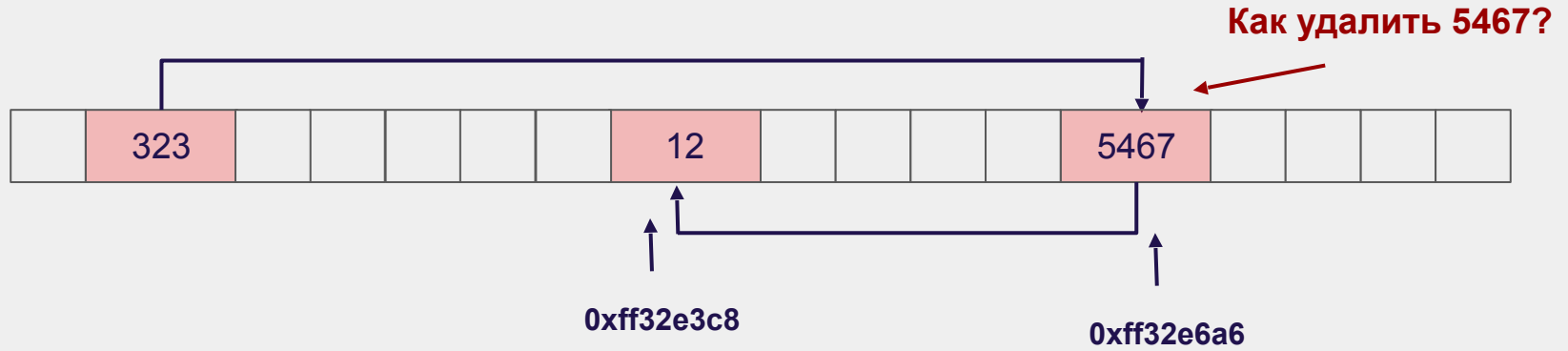


Оперативная память



Нужно удалить этот лист

Оперативная память

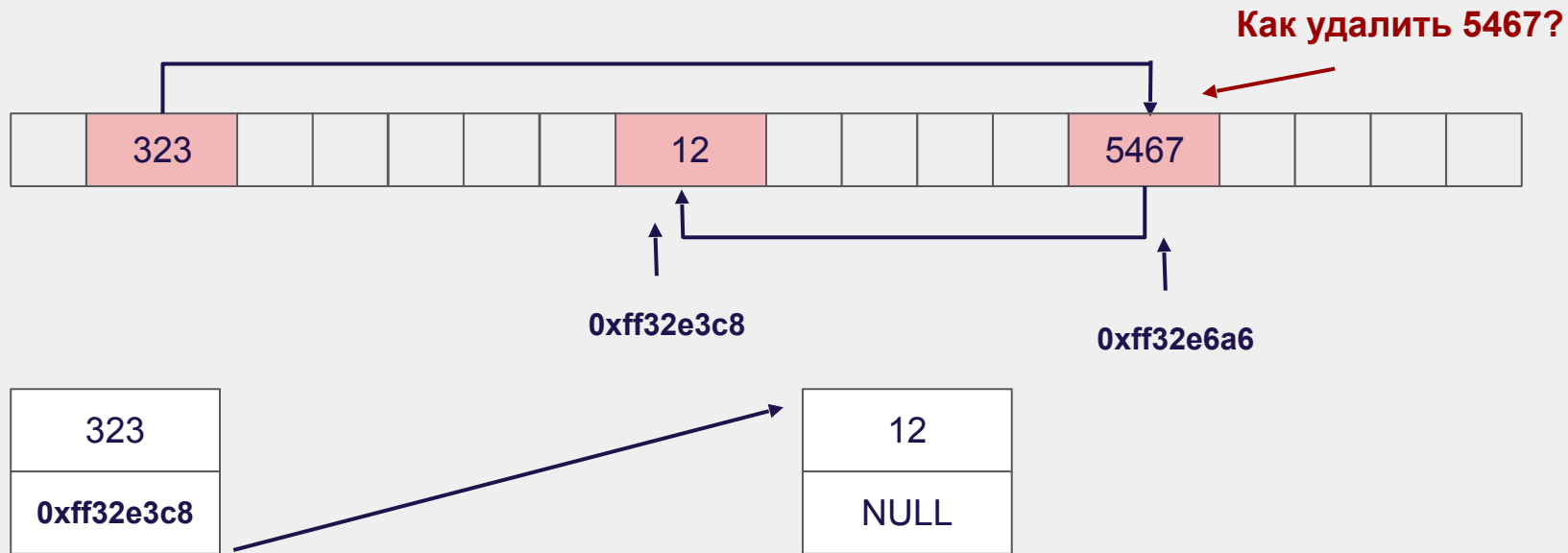


323
0xff32e6a6

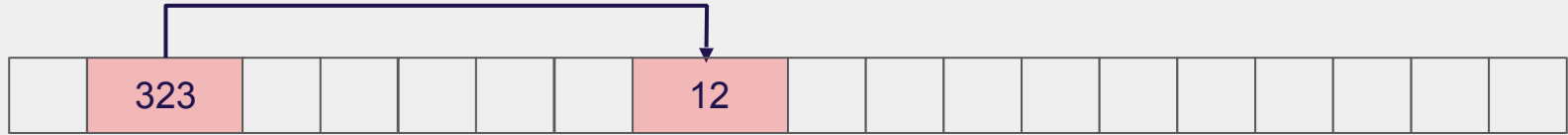
12
NULL

Обновить адрес next

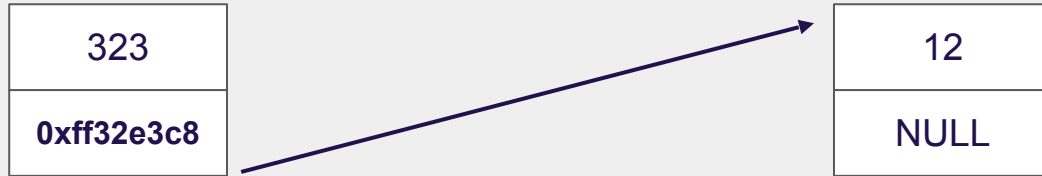
Оперативная память



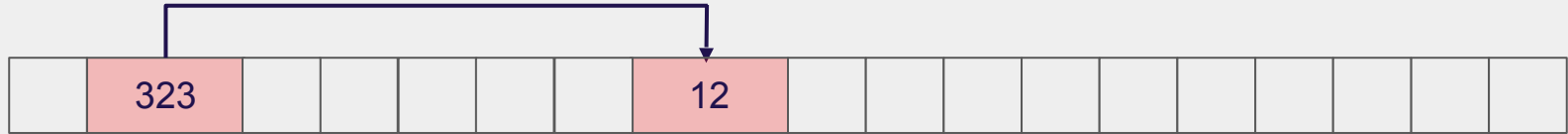
Оперативная память



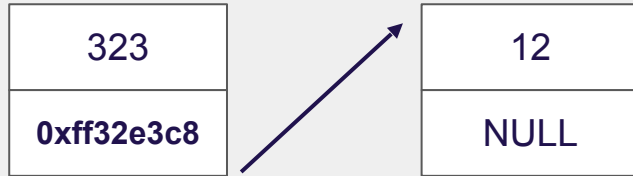
0xff32e3c8



Оперативная память

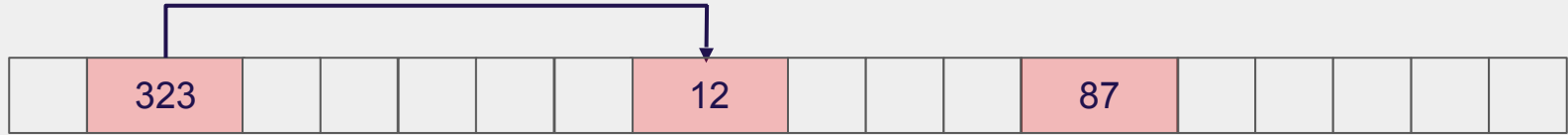


0xff32e3c8

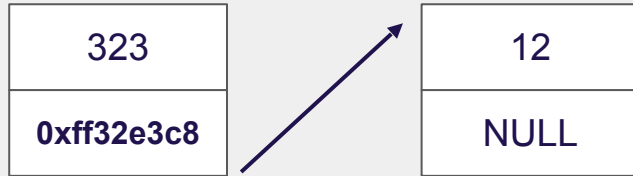


**Добавление элемента
в конец**

Оперативная память

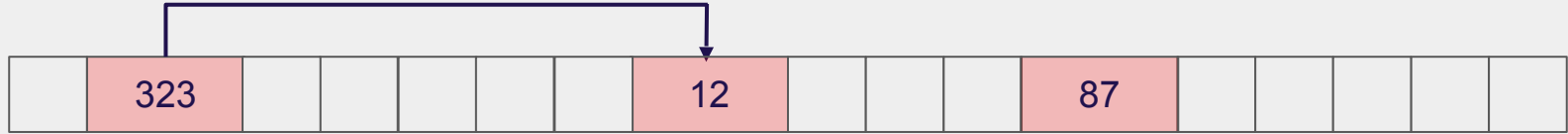


↑
0xff32e3c8

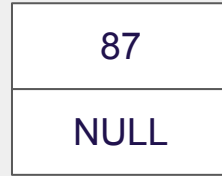
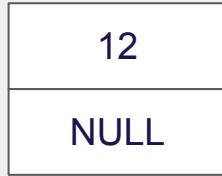
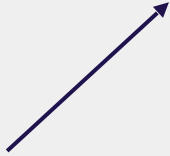


Как добавить 87?

Оперативная память

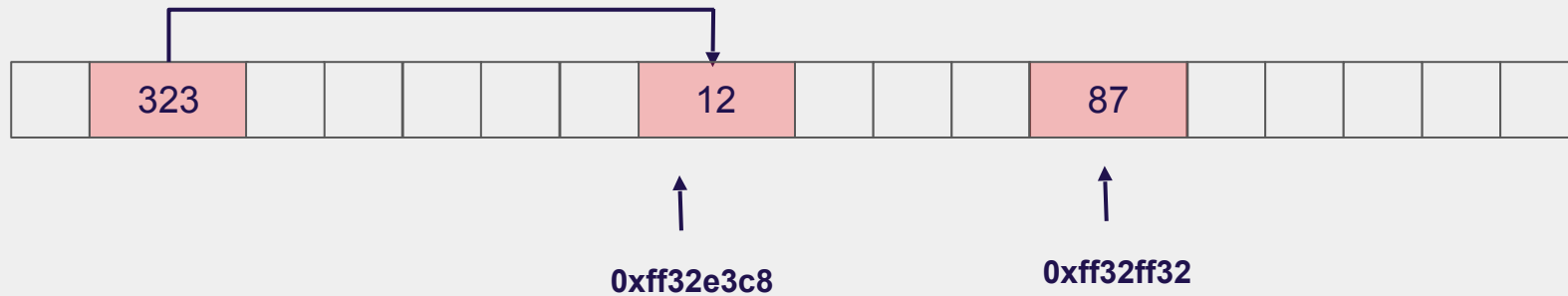


0xff32e3c8



Нужно создать лист

Оперативная память



**И переставить
указатель**

Реализация

Список

```
struct Node
{
    int value;
    Node* next;
};
```

Создадим **структуру листа** этого списка

Список

```
struct Node
{
    int value;
    Node* next;
};

class List
{
    Node* head = 0;
};
```

Создадим **класс** для списка

Список

```
struct Node
{
    int value;
    Node* next;
};

class List
{
    Node* head = 0;

public:
    void pushBack(int value);
};
```

Реализуем для **списка** функцию
вставки в конец

Список

```
void List::pushBack(int value)
{
```

```
};
```

Реализация метода pushBack

Список

```
void List::pushBack(int value)
{
    Node* newNode = new Node;
    newNode -> value = value;
    newNode -> next = 0;
```

```
};
```

Создадим новый лист списка

Список

```
void List::pushBack(int value)
{
    Node* newNode = new Node;
    newNode -> value = value;
    newNode -> next = 0;

    if (!head) {
        head = newNode;
        return;
    }

};
```

Проверим случай когда список пуст

Список

```
void List::pushBack(int value)
{
    Node* newNode = new Node;
    newNode -> value = value;
    newNode -> next = 0;

    if (!head) {
        head = newNode;
        return;
    }

    Node* last = head;

    while (last -> next)
        last = last->next;

    last -> next = newNode;
};
```

Если не пуст то найдем конец списка и
вставим элемент после last

Список

```
class List
{
    Node* head = 0;

public:

    void pushBack(int value);
    void pushFront(int value);
    int popFront();
    int popBack();

    void insert(int index, int value);
    void delete(int index);
    int length();
    void clear();

    int* operator[](int index);
};
```

Все методы списка

Задание

Реализовать оставшиеся методы списка

Способы хранения

Хранение множества объектов:

- Массив
- Список

Архитектура приложения

Пользователь

